VCCA

Vertical Cleaner Clean Architecture

Documentation Summary

C# · .NET 8 · Blazor Server · SQL Server · MediatR · SAP · NetSuite · Odoo

Maintained by Ian Nicolas Antonio · 2026

📋 Vertical Cleaner Clean Architecture

Topics to be Discussed

Big Picture · The Four Layers
Domain · Application Layers
Infrastructure · Presentation Layers
Pipeline · Auth · SAP · NetSuite · Odoo
Conventions

⚡ The #1 Rule

> Every layer has a job. Don't skip layers.

```
User clicks a button
  → Blazor Component
    → Web Repository
      → Web Handler  (IMediator.Send)
        → Application Handler
          → Infrastructure / Repository
            → SQL Server
```

This chain **always** stays intact — every feature follows this exact path.

🌐 The Big Picture

Part 1

What is VCCA and how do the pieces fit?

PART 01 OF 12

🏛️ What Is VCCA?

Vertical Cleaner Clean Architecture combines two patterns:

Influence	What It Contributes
Clean Architecture	Layered separation · strict dependency rule
Vertical Slice Architecture	Feature-cohesive folder grouping

Platform

Language	**C# / .NET 8**
Frontend	**Blazor Server** (SignalR-based)
Database	**Microsoft SQL Server**
External	**SAP** via B1SLayer · **NetSuite** REST API · **Odoo** JSON-RPC

🍽️ The Restaurant Analogy

Restaurant	System	Responsibility
👤 Customer	Browser / User	Makes a request
👨‍🍳 Waiter	Blazor UI (Presentation)	Takes order, delivers result
🧑‍🍳 Head Chef	Application Layer	Decides *how* to fulfil it
📖 Recipe Book	Domain Layer	Defines *what the rules are*
🏭 Kitchen Equipment	Infrastructure	Stores & retrieves data
🚚 Outside Vendor	SAP Integration	External data supplier

> **The chef never goes grocery shopping himself.**
> Business logic never touches the database directly.

🗺️ Solution Map

```
your-project/
├── core/
│   ├── domain/
│   │   └── Domain.Entities          ← Business rules & data shapes
│   └── application/
│       ├── Application.DTOs         ← Data packets between layers
│       └── Application.UseCases     ← "What the system can do"
├── shared/
│   └── Shared                       ← Helpers, extensions
└── src/
    ├── infrastructure/database/
    │   ├── Database.Libraries       ← Abstract interfaces
    │   └── Database.MsSql           ← EF Core · SQL Server
    ├── integration/
    │   ├── Integration.Sap          ← SAP HTTP client
    │   ├── Integration.NetSuite     ← NetSuite REST + SuiteQL
    │   └── Integration.Odoo         ← Odoo JSON-RPC
    └── presentation/
        └── Web.BlazorServer         ← The UI users see
```

🧱 The Four Layers

DOMAIN	– Domain.Entities	Business rules · no external deps
APPLICATION	– Application.*	Orchestrates use cases
INFRASTRUCTURE	– Database · SAP	Technical implementations
PRESENTATION	– Web.BlazorServer	What users see

Golden Rule of Dependencies

- ✅ Presentation → Application → Domain
- ✅ Application → Infrastructure *(via interfaces only)*
- ❌ Domain must **never** know about any outer layer
- ❌ Application must **never** reference Presentation

🔧 Tech Stack at a Glance

Library	Purpose	Used In
MediatR	Decoupled command/query dispatch	Application · Web
EF Core	ORM for SQL Server	Database.MsSql
Dapper	Lightweight raw SQL	Database.MsSql
Mapster	Entity ↔️ DTO mapping	Application.UseCases
Ardalis.Guards	`Guard.Against.*` validation	Domain · Application
Radzen Blazor	Rich UI components	Web.BlazorServer
Blazor Server	Server-side UI via SignalR	Web.BlazorServer
B1SLayer	SAP REST integration	Integration.Sap
NetSuite REST	SuiteQL · OAuth2/JWT (PS256)	Integration.NetSuite
Odoo JSON-RPC	`execute_kw` · uid/password per-request	Integration.Odoo

🧬 Domain Layer

Part 2 — Layer 1

The heart of the system · pure business rules

PART 02 OF 12

🧬 Domain Layer — Overview

`core/domain/Domain.Entities` · **zero external dependencies**

❌ No EF Core · No HTTP · No MediatR* · No application workflows

Artifact	Description	Example
Entities	Main data objects	`OrderDEM`, `UserDEM`
Value Objects	Immutable, identity-free	`MoneyVO`, `AddressVO`
Enums	System-wide enumerations	`OrderStatus`
Domain Events	Fired on important actions	`OrderApprovedEvent`
Aggregate Roots	Entities owning child data	`Order` → `OrderLines`

> * Domain does reference MediatR for `INotification` — a known architectural debt.

🧬 EntityDEM — The Base Class

```
public abstract class EntityDEM : IDomainEntity
{
    public Guid Id { get; private set; }    // Auto-generated

    protected EntityDEM()
    {
        Id = Guid.NewGuid();
        DomainEvents = [];
    }
}
```

Inheritance Chain

```
EntityDEM
  └── AuditableDEM    ← adds CreatedBy, CreatedDate, UpdatedBy, UpdatedDate
        └── OrderDEM  ← your entity
```

> Use `AuditableDEM` for anything the business needs to track changes on.

🧬 Entity Rules 1 & 2

Rule 1 — Private setters protect state

```
public class OrderDEM : AuditableDEM
{
    public string      CustomerName { get; private set; }
    public OrderStatus Status       { get; private set; }
    public decimal     Total        { get; private set; }
}
```

Rule 2 — Static factory: the only way to create

```
public static OrderDEM Create(string customerName, decimal total)
    => new(customerName, total);

private OrderDEM(string customerName, decimal total)
{
    CustomerName = Guard.Against.NullOrEmpty(customerName, nameof(customerName));
    Total        = Guard.Against.NegativeOrZero(total, nameof(total));
    Status       = OrderStatus.Pending;
}
```

🧬 Entity Rule 3 — All Changes Through Methods

```
public class OrderDEM : AuditableDEM
{
    public string CustomerName { get; private set; }
    public OrderStatus Status  { get; private set; }

    private readonly List<OrderLineDEM> _lines = [];
    public IReadOnlyCollection<OrderLineDEM> Lines => _lines.AsReadOnly();

    private OrderDEM() { }   // Required by EF Core
    private OrderDEM(string customerName)
    {
        CustomerName = Guard.Against.NullOrEmpty(customerName, nameof(customerName));
        Status = OrderStatus.Pending;
    }
    public static OrderDEM Create(string customerName) => new(customerName);

    public void AddLine(OrderLineDEM line)
    {
        Guard.Against.Null(line, nameof(line));
        _lines.Add(line);
    }
    public void Approve()
    {
        if (Status ≠ OrderStatus.Pending)
            throw new InvalidOperationException("Only pending orders can be approved.");
        Status = OrderStatus.Approved;
    }
}
```

Ardalis.Guards

`Guard.Against.*` — expressive, consistent input validation

```
Guard.Against.Null(value, nameof(value));
// Must not be null

Guard.Against.NullOrEmpty(text, nameof(text));
// Must not be null or ""

Guard.Against.NullOrWhiteSpace(text, nameof(text));
// Must not be null, "", or "   "

Guard.Against.NegativeOrZero(number, nameof(number));
// Must be > 0

Guard.Against.Negative(number, nameof(number));
// Must be ≥ 0

Guard.Against.OutOfRange(value, nameof(value), min, max);
// Must be within range
```

Use in constructors and domain methods to enforce invariants at the boundary.

💎 Value Objects

Represent a concept **by value**, not by identity. Equal if all values match.

```
public class MoneyVO
{
    public decimal Amount   { get; }
    public string  Currency { get; }

    public MoneyVO(decimal amount, string currency)
    {
        Amount   = Guard.Against.Negative(amount, nameof(amount));
        Currency = Guard.Against.NullOrEmpty(currency, nameof(currency));
    }
    public override bool Equals(object? obj)
        => obj is MoneyVO o && Amount == o.Amount && Currency == o.Currency;
}
```

	Entity	Value Object
Identity	Has `Id` (Guid)	No identity
Equality	By `Id`	By value
Mutability	Methods change state	Immutable

📣 Domain Events

Notify other parts of the system — **without tight coupling**.

```
// 1. Define
public class OrderApprovedEvent : DomainBaseEvent
{
    public Guid OrderId { get; }
    public OrderApprovedEvent(Guid id) => OrderId = id;
}

// 2. Fire from entity
public void Approve()
{
    Status = OrderStatus.Approved;
    DomainEvents.Add(new OrderApprovedEvent(Id));
}

// 3. Handle in Application layer
public class OrderApprovedHandler : INotificationHandler<OrderApprovedEvent>
{
    public async Task Handle(OrderApprovedEvent e, CancellationToken ct)
    { /* send email, update SAP, etc. */ }
}
```

> ⚠ **Known Debt:** Domain imports MediatR for `INotification`. Correct fix: local `IDomainEvent` interface adapted in Application layer.

📋 Enums & Aggregate Root Warning

```
// Domain.Entities/Enums/OrderStatus.cs
public enum OrderStatus
{
    [Description("Pending")]   Pending,
    [Description("Approved")]  Approved,
    [Description("Cancelled")] Cancelled
}
```

> ⚠️ **Aggregate Root Warning**
>
> In proper DDD, only the aggregate root should be modified directly.
> Our codebase **does not always enforce this**.
> Before writing a handler that touches child entities — **check how that specific aggregate is structured first.**
> Do not fix this without explicit tech lead instruction.

⚙ Application Layer

Part 3 — Layer 2

The orchestrator · coordinates use cases

PART 03 OF 12

⚙ Application Layer — Overview

`Application.UseCases` + `Application.DataTransferObjects`

Artifact	Purpose
MediatR Handlers	Process a command or query
Commands	Requests to *change* something
Queries	Requests to *read* something
IXxxService interfaces	Contracts Infrastructure must implement
Mapster Profiles	Domain entity ↔ DTO conversion rules
DTOs	Simple data packets, no behaviour

❌ No Blazor/UI code · No raw SQL/EF DbContext calls · No ViewModels

⚙ Application Folder Structure

```
Application.UseCases/
├── Commands/
│   └── Orders/
│       ├── CreateOrderCommand.cs   ← what data the request needs
│       └── CreateOrderHandler.cs   ← the use case logic
└── Queries/
    └── Orders/
        ├── GetOrderByIdQuery.cs
        └── GetOrderByIdHandler.cs

Application.DataTransferObjects/
└── Orders/
    └── OrderDto.cs                 ← plain data, no behaviour
```

> 💡 Vertical slice principle — each feature folder contains both the command/query **and** its handler together.

⚙ Application Handler — Example

```
public class CreateOrderHandler : IRequestHandler<CreateOrderCommand, OrderDto>
{
    private readonly IOrderRepository _repo;
    public CreateOrderHandler(IOrderRepository repo) => _repo = repo;

    public async Task<OrderDto> Handle(
        CreateOrderCommand request, CancellationToken ct)
    {
        // 1. Create domain entity
        var order = OrderDEM.Create(request.CustomerId, request.Items);

        // 2. Persist via repository (Infrastructure does the actual saving)
        await _repo.AddAsync(order);

        // 3. Map to DTO — NEVER return the raw entity
        return order.Adapt<OrderDto>();
    }
}
```

Three responsibilities: Load/create → orchestrate domain methods → return DTO

⚙ Commands vs Queries

```
// Commands – change something
public record CreateOrderCommand(
    Guid CustomerId, List<OrderItemDto> Items
) : IRequest<OrderDto>;
```

```
// Queries – read something
public record GetOrderByIdQuery(Guid Id) : IRequest<OrderDto?>;

public class GetOrderByIdHandler : IRequestHandler<GetOrderByIdQuery, OrderDto?>
{
    public async Task<OrderDto?> Handle(
        GetOrderByIdQuery q, CancellationToken ct)
        => (await _repo.GetByIdAsync(q.Id))?.Adapt<OrderDto>();
}
```

	Command	Query
Purpose	Change state	Read state
Returns	DTO or void	DTO or null

📦 DTOs — Data Transfer Objects

Simple data packets — **no business logic, no behaviour.**

```
public class OrderDto
{
    public Guid     Id           { get; set; }
    public string   CustomerName { get; set; }
    public decimal  Total        { get; set; }
    public string   Status       { get; set; }
    public DateTime CreatedDate  { get; set; }
}
```

Concern	Domain Entity	DTO
Business rules	✅ Enforced	❌ None
Private setters	✅ Protected	❌ Public
Cross-layer safe	❌ Risky	✅ Safe
Serialisable	❌ Complex	✅ Simple

> **Never** return a Domain entity from a handler. Always `Adapt<OrderDto>()` first.

Infrastructure Layer

Part 4 — Layer 3

The technical details · data access · external calls

PART 04 OF 12

🗄️ Infrastructure Layer — Overview

Implements the contracts defined by Application.

Project	Contents
`Database.Libraries`	`IRepository<T>`, EF Core base classes
`Database.MsSql`	`AppDbContext`, migrations, `XxxRepository`, entity configs
`Integration.Sap`	HTTP client methods for SAP APIs
`Integration.NetSuite`	REST + SuiteQL client for NetSuite
`Integration.Odoo`	JSON-RPC client for Odoo

❌ No business rules · No UI code · No DTO definitions

Two Repository Types — Both Intentional, Don't Consolidate

Type	Use When
`IRepository<T>`	Simple CRUD
`IOrderRepository`	Complex entity-specific queries

🗄️ The Repository Pattern

> A repository is a **filing cabinet** for one type of data. You ask it for orders — it gives you orders. You don't care *how* it stores them.

```
// Generic – simple saves/reads
await _repository.AddAsync(order);
await _repository.GetByIdAsync(id);

// Specific – complex, entity-specific queries
await _orderRepository.GetOrdersWithLineItemsByCustomerAsync(customerId);
await _orderRepository.GetPendingOrdersOlderThanAsync(
    DateTime.UtcNow.AddDays(-7));
```

Rule: Prefer specific repositories for anything beyond basic CRUD.

🗄️ Entity Configuration

```
// ✅ CORRECT — configuration in Database.MsSql, not in Domain
public class OrderConfiguration : IEntityTypeConfiguration<OrderDEM>
{
    public void Configure(EntityTypeBuilder<OrderDEM> builder)
    {
        builder.ToTable("Orders");
        builder.HasKey(x => x.Id);
        builder.Property(x => x.CustomerName).HasMaxLength(200).IsRequired();
    }
}
```

```
// ❌ WRONG — EF attributes polluting the Domain entity
public class OrderDEM : EntityDEM
{
    [Required]        // ❌ EF attribute in Domain!
    [MaxLength(200)]  // ❌ Domain must stay pure
    public string CustomerName { get; private set; }
}
```

Keep the Domain layer **free of EF Core dependencies.**

🗄️ Query Rules

Always `.AsNoTracking()` for reads

```
// ✅ Faster — EF won't track changes, uses less memory
var orders = await _context.Orders
    .AsNoTracking()
    .Where(o => o.CustomerId == customerId)
    .ToListAsync();
```

Always async · Never block

```
// ✅                                    // ❌
await _context.Orders.ToListAsync()     _context.Orders.ToList()
```

Parameterised queries — never string interpolation

```
// ✅ EF handles parameterisation automatically
.Where(o => o.CustomerName == customerName)

// ❌ SQL injection risk!
.FromSqlRaw($"SELECT * FROM Orders WHERE Name = '{customerName}'")
```

🗄️ Database Migrations

Always run from the **solution root**.

```
# Create
dotnet ef migrations add AddOrderStatusColumn \
  --project src/infrastructure/database/Database.MsSql \
  --startup-project src/presentation/Web.BlazorServer

# Apply
dotnet ef database update \
  --project src/infrastructure/database/Database.MsSql \
  --startup-project src/presentation/Web.BlazorServer
```

Rule	Why
✅ Review generated file before applying	EF can produce unexpected drops
✅ Descriptive names e.g. `AddOrderStatusColumn`	Readable history
✅ Add a new migration to fix mistakes	Don't edit applied ones
❌ Never modify an applied migration	Breaks history on other envs

Presentation Layer

Part 5 — Layer 4

What users see · Blazor Server UI

PART 05 OF 12

🖥️ Presentation Layer — Overview

`src/presentation/Web.BlazorServer`

Sole job: **receive input → Application → display result**

```
Web.BlazorServer/
├── Components/
│   ├── Base/            ← BaseComponent · BaseForm<TItem>
│   ├── Layout/          ← ProtectedLayout
│   └── Pages/[Feature]  ← OrderPage.razor + .razor.cs
├── Handlers/
│   ├── Repositories/    ← IXxxHandler interfaces
│   └── Implementations/ ← Thin MediatR dispatchers
├── Services/            ← Alert · Busy · Toast
└── ViewModels/          ← UI-specific display models
```

❌ No business logic · No direct DB access · No direct SAP calls

🖥️ BaseComponent — The Foundation

Every component must inherit `BaseComponent`.

`@inherits BaseComponent` ✅ Always required

Already injected — never re-inject these

Service	Available As
`AppActionFactory`	`AppActionFactory`
`IBusyService`	`BusyService`
`IToastService`	`ToastService`
`IAlertService`	`AlertService`
`NavigationManager`	`NavManager`
`DialogService`	`DialogService`
`IJSRuntime`	`JSRuntime`
`ICurrentUserService`	`CurrentUser`

🖥️ BaseForm<TItem> — CVU Pages

CVU (Create/View/Update) pages inherit `BaseForm<TItem>` instead.

```
BaseComponent
  └── BaseForm<TItem>   ← CVU pages inherit this
```

Provides: `FormData`, `FormDataClone`, `RadzenTemplateForm` integration, unsaved-change tracking, submit/cancel lifecycle.

```
public partial class OrderCVU : BaseForm<OrderVM>
{
    protected override async Task InitializeEditing()
    { /* load existing record into FormData */ }

    protected override async Task HandleSubmit()
    { /* save FormData via handler */ }

    protected override Task CancelEditing()
    {
        AdaptToForm();          // restore snapshot
        NavManager.NavigateTo("/orders");
        return Task.CompletedTask;
    }
}
```

💻 Code-Behind Convention

Two files per page — **markup only** vs **all logic**.

```
{{!-- OrderPage.razor – markup only --}}
@page "/orders"
@inherits BaseComponent

@if (_isLoading) { <p>Loading ... </p> }
else { <AppDataGrid Items="@_orders" /> }
```

```
// OrderPage.razor.cs – all logic
public partial class OrderPage : BaseComponent
{
    [Inject] private IOrderHandler OrderHandler { get; set; } // ✅ Handler only

    private List<OrderVM> _orders = new();

    protected override async Task OnInitializedAsync()
    {
        var action = await AppActionFactory.RunAsync(
            async () => await OrderHandler.GetAllOrdersAsync(),
            AppActionOptionPresets.Loading(AppActions.GetAllOrders.GetDescription()));

        action.OnSuccess(r => { _orders = r?.Adapt<List<OrderVM>>() ?? []; });
    }
}
```

🖥️ ViewModels vs DTOs

	`XxxVM` (ViewModel)	`XxxDto` (DTO)
Location	`Web.BlazorServer/ViewModels/`	`Application.DataTransferObjects/`
Purpose	Display in UI	Data contract between layers
Extra fields	`IsSelected`, formatted strings	Plain data only
Returned from handlers?	❌ Never	✅ Yes

```
// ViewModel — UI-specific extras
public class OrderVM
{
    public Guid   Id           { get; set; }
    public string CustomerName { get; set; }
    public decimal Total       { get; set; }
    public bool   IsSelected   { get; set; }                  // UI state
    public string FormattedTotal => $"₱{Total:N2}";           // Display helper
}
```

Mapping in the component: `dto.Adapt<OrderVM>()`

⚡ Async Rules — Critical for Blazor Server

Blazor Server runs on a **SignalR connection** — blocking it **freezes the UI for all users.**

```
// ✅ Always async
protected override async Task OnInitializedAsync()
    => _data = await Repository.GetDataAsync();

// ❌ NEVER — these block the thread and cause deadlocks
var data = Repository.GetDataAsync().Result;
Repository.GetDataAsync().Wait();
Repository.GetDataAsync().GetAwaiter().GetResult();
```

`StateHasChanged()` — only when needed

```
// ✅ Needed — state changed outside Blazor's event cycle
await InvokeAsync(StateHasChanged);   // e.g. inside a Timer callback

// ❌ Not needed — Blazor re-renders automatically after @onclick
private async Task HandleClickAsync()
    => _items = await Repository.GetItemsAsync(); // No StateHasChanged needed
```

🎨 Radzen Forms — Use `RadzenTemplateForm`

```
{{!-- ❌ Never Blazor's EditForm --}}
<EditForm Model="@_order"><DataAnnotationsValidator /></EditForm>

{{!-- ✅ Always RadzenTemplateForm --}}
<RadzenTemplateForm TItem="OrderVM" Data="@_order" Submit="HandleSaveAsync">

    <RadzenFormField Text="Customer Name" Style="width:100%">
        <RadzenTextBox @bind-Value="@_order.CustomerName" />
    </RadzenFormField>

    <RadzenFormField Text="Total Amount" Style="width:100%">
        <RadzenNumeric TValue="decimal" @bind-Value="@_order.Total" />
    </RadzenFormField>

    <RadzenFormField Text="Status" Style="width:100%">
        <RadzenDropDown @bind-Value="@_order.StatusId"
            Data="@_statusOptions" TextProperty="Label" ValueProperty="Value" />
    </RadzenFormField>

    <RadzenButton ButtonType="ButtonType.Submit" Text="Save" />
    <RadzenButton ButtonStyle="ButtonStyle.Light" Text="Cancel"
                  Click="HandleCancelAsync" />
</RadzenTemplateForm>
```

🔄 The Request Pipeline

Part 6

How a button click becomes a database query

PART 06 OF 12

🔄 The Full Request Pipeline

```
👤 User clicks a button
   │
   ▼ [.razor.cs]
Blazor Component          – handles UI events
   │ calls
   ▼ [/Repositories/Feature/]
Web Repository            – groups related operations
   │ calls
   ▼ [/Handlers/Feature/]
Web Handler               – calls IMediator.Send()
   │ IMediator.Send(command/query)
   ▼ [Application.UseCases]
Application Handler       – orchestrates the use case
   │ calls
   ▼ [Database.Libraries interface → Database.MsSql impl]
IXxxRepository            – data access contract
   │
   ▼ [Database.MsSql]
AppDbContext              – EF Core → SQL Server
```

🔄 Pipeline — Step by Step

```
// Step 1 — Component triggers
private async Task HandleSaveAsync()
    => await OrderRepository.SaveOrderAsync(ViewModel);

// Step 2 — Web Repository packages
public async Task<OrderDto> SaveOrderAsync(OrderVM vm)
    => await _handler.SaveOrderAsync(new SaveOrderCommand(vm.Id, vm.Items));

// Step 3 — Web Handler dispatches (ONLY place IMediator.Send is called)
public async Task<OrderDto> SaveOrderAsync(SaveOrderCommand cmd)
    => await _mediator.Send(cmd);

// Step 4 — Application Handler processes
public async Task<OrderDto> Handle(SaveOrderCommand req, CancellationToken ct)
{
    var order = await _repo.GetByIdAsync(req.Id);
    order.UpdateItems(req.Items);      // domain method enforces rules
    await _repo.UpdateAsync(order);
    return order.Adapt<OrderDto>();    // map to DTO — never return entity
}
```

🔄 Return Journey & Anti-Patterns

```
DB → Repository → AppHandler (maps DTO) → MediatR
   → WebHandler → WebRepository → Component (maps ViewModel)
```

```
// ❌ Injecting IMediator directly into a component
@inject IMediator Mediator
await Mediator.Send(new SomeCommand());   // skips Repository/Handler chain!

// ❌ Calling the database from a component
@inject AppDbContext DbContext
var orders = await DbContext.Orders.ToListAsync();  // Infrastructure in Presentation!

// ❌ Business logic in a Web Handler
public async Task<OrderDto> SaveAsync(SaveOrderCommand cmd)
{
    if (cmd.Items.Count == 0)
        throw new Exception("...");  // ❌ logic doesn't belong here
    return await _mediator.Send(cmd);
}
```

✅ New Feature Checklist

Build files in this order every time:

Step	Layer	Task
1	Domain	Entity exists? Create/update if not
2	Application	Define the Command or Query
3	Application	Write the Handler use case logic
4	Application	Define the DTO
5	Presentation	Web Handler (thin `IMediator.Send` dispatcher)
6	Presentation	Web Repository (group related operations)
7	Presentation	ViewModel (UI-specific display model)
8	Presentation	Blazor Component (page the user sees)

🔐 Authentication & Authorization

Part 7

Login · cookies · permissions

PART 07 OF 12

🔐 Authentication Overview

Custom **Microsoft Cookie Authentication** — not JWT, not ASP.NET Identity.

> 💡 **Wristband analogy:** When you log in, you get a wristband (cookie) stamped with your roles and permissions. Every page checks the wristband — no round-trip to the front desk.

1. User submits credentials (Blazor login page)
   ↓
2. login.js POSTs to AuthenticationController
   ↓
3. Controller → MediatR → Application validates
   ↓
4. Application returns UserDTO
   ↓
5. Controller builds ClaimsPrincipal → issues auth cookie
   ↓
6. All subsequent requests authenticated via that cookie

🔐 Claims & Permission-Based Auth

Claims Written at Login

Claim	Value
`Id`	User GUID
`Name`	Full name
`RoleId`	Role GUID
`Role`	Role name
`Email`	Email address
`Permissions`	Serialised permission string

> ⚠️ Changes take effect at **next login** — claims are not refreshed mid-session.

```
// Resolves to policy: "permission.ORDERS.CREATE"
[AppAuthorization("CREATE", "ORDERS")]
public partial class CreateOrderPage : BaseComponent { ... }
```

`AppPolicyProvider → PermissionRequirement → cookie claim → ✅ / ❌`

🔐 Securing Pages and Components

```
{{!-- Protect an entire page --}}
@page "/orders"
@layout ProtectedLayout
@inherits BaseComponent
```

```
{{!-- Permission check in markup --}}
<AuthorizeView Policy="permission.ORDERS.DELETE">
    <Authorized>
        <RadzenButton Text="Delete" />
    </Authorized>
    <NotAuthorized>
        <p>You don't have permission to delete orders.</p>
    </NotAuthorized>
</AuthorizeView>
```

Rules — Never Break

- Never hardcode credentials, signing keys, or secrets anywhere
- Use `AppAuthorizationAttribute` or `AuthorizeView` for UI-level access control
- Do **not** add auth logic to Application or Domain layers
- Do **not** re-implement cookie generation outside `AuthenticationController`

SAP Integration

Part 8

Talking to the outside world

PART 08 OF 12

🏭 SAP Integration — Overview

`src/integration/Integration.Sap` · all calls **outbound only**

> The Rule: All SAP HTTP calls must go through `Integration.Sap`. No exceptions.

```
Blazor Component → Web Repository → Web Handler
  → IMediator.Send() → Application Handler
    → Integration.Sap  ← SAP calls ONLY here
      → SAP External API
```

Why This Structure?

- SAP API changes → update one project only
- All SAP error handling is centralised
- Easy to mock in tests
- Clear visibility into all external calls

🏭 Adding a New SAP Endpoint

```
// Step 1 – HTTP client method in Integration.Sap
public async Task<SapOrderResponse> GetOrderFromSapAsync(string sapNo)
{
    var response = await _httpClient.GetAsync($"/api/orders/{sapNo}");
    response.EnsureSuccessStatusCode();
    return await response.Content.ReadFromJsonAsync<SapOrderResponse>();
}
```

```
// Step 2 – Query + Handler in Application.UseCases
public record GetSapOrderQuery(string SapOrderNumber) : IRequest<SapOrderDto>;

public class GetSapOrderHandler : IRequestHandler<GetSapOrderQuery, SapOrderDto>
{
    private readonly ISapOrderService _sap;
    public async Task<SapOrderDto> Handle(GetSapOrderQuery q, CancellationToken ct)
        => (await _sap.GetOrderFromSapAsync(q.SapOrderNumber)).Adapt<SapOrderDto>();
}
```

Step 3 — wire up through the normal Web Repository → Web Handler pipeline.

NetSuite Integration

Part 9

Connecting to NetSuite ERP · REST API + SuiteQL

PART 09 OF 12

🌐 NetSuite — Overview

`Integration.NetSuite/` · all calls **outbound only**

> The Rule: All NetSuite calls must go through `Integration.NetSuite`. No exceptions.

Mode	API	Used For
REST Record API	HTTP CRUD	Creating / updating NetSuite records
SuiteQL	SQL-like query via REST	Reading data sets (e.g. pending POs)

```
Component → Web Handler → IMediator.Send()
  → Application Handler → IXxxNSIntegration
    → Integration.NetSuite  ← NetSuite calls ONLY here
      → NetSuite External API
```

🌐 NetSuite — Auth & Setup

Authentication: OAuth2 client credentials with **JWT assertion (PS256 / RSA-PSS)** — no passwords. `NetSuiteConnection` (Singleton) handles JWT signing, token exchange, and auto-refresh transparently.

DI Registration

```
builder.Services.AddNetSuiteServicesIntegration();
// → INetSuiteConnection  (Singleton) – auth + token caching
// → INetSuiteActions     (Transient) – HTTP + SuiteQL dispatch
```

Credentials — always via env vars, never in code

Variable	Purpose
`ACCOUNT_ID`	NetSuite account identifier
`NETSUITE_CERTIFICATE_ID`	OAuth certificate ID
`NETSUITE_CONSUMER_KEY`	Consumer key
`PRIVATEKEY_PATH`	Path to RSA private key (PKCS#8 PEM, min 3072 bits)

🌐 Creating a NetSuite Integration — 4 Steps

1 Add SuiteQL script → `Integration.NetSuite/NSScripts/NS_{Feature}_{Action}.sql`

2 Define interface in `Application.UseCases/Repositories/Integration/`:

```
public interface IReceivingNSIntegration
{
    Task<IEnumerable<PurchaseOrderDTO>> GetPendingReceiptPOsAsync(int limit = 0, int offset = 0);
}
```

3 Implement in `Integration.NetSuite/Implementations/` — always `internal`:

```
internal class ReceivingIntegration(INetSuiteActions nsActions) : IReceivingNSIntegration
{
    public async Task<IEnumerable<PurchaseOrderDTO>> GetPendingReceiptPOsAsync( ... )
        => await nsActions.QueryAsync<PurchaseOrderDTO>("NS_PurchaseOrder_Get_PendingReceipt");
}
```

4 Register → `services.TryAddTransient<IReceivingNSIntegration, ReceivingIntegration>()`

🌐 NetSuite → Application.UseCases Pipeline

```
// MediatR Query — always IRequest, NEVER ITransactionalRequest
public record GetPendingReceiptPOsQry(int Limit, int Offset)
    : IRequest<IEnumerable<PurchaseOrderDTO>>;

public class GetPendingReceiptPOsQryHandler(IReceivingNSIntegration receiving)
    : IRequestHandler<GetPendingReceiptPOsQry, IEnumerable<PurchaseOrderDTO>>
{
    public Task<IEnumerable<PurchaseOrderDTO>> Handle(
        GetPendingReceiptPOsQry req, CancellationToken ct)
        => receiving.GetPendingReceiptPOsAsync(req.Limit, req.Offset);
}
```

Web Handler and component follow the **same pattern as any other feature.**

```
Component → IReceivingHandler → IMediator.Send(GetPendingReceiptPOsQry)
  → GetPendingReceiptPOsQryHandler → IReceivingNSIntegration
    → ReceivingIntegration → INetSuiteActions → NetSuite API
```

Odoo Integration

Part 10

Connecting to Odoo ERP · JSON-RPC

PART 10 OF 12

Odoo — Overview

`Integration.Odoo/` · all calls **outbound only**

> The Rule: All Odoo calls must go through `Integration.Odoo`. No exceptions.

Method	When to Use
`SearchReadAsync`	Fetch a list of records from a model
`SearchReadOneAsync`	Fetch the first matching record
`CreateAsync`	Create a new record — returns the new record ID
`WriteAsync`	Update existing records by their IDs
`UnlinkAsync`	Delete records by their IDs

```
Component → Web Handler → IMediator.Send()
  → Application Handler → IXxxOdooIntegration
    → Integration.Odoo  ← Odoo calls ONLY here
      → Odoo External API (JSON-RPC)
```

Odoo — Auth & Setup

Authentication: credentials (`db`, `uid`, `password`) embedded in every JSON-RPC call — no token exchange.

DI Registration

```
builder.Services.AddOdooServicesIntegration();
// → IOdooConnection  (Singleton) — holds credentials
// → IOdooActions     (Transient) — HTTP + JSON-RPC dispatch
```

Credentials — always via env vars, never in code

Variable	Purpose
`ODOO_BASE_URL`	Odoo server base URL
`ODOO_DATABASE`	Database name
`ODOO_UID`	User ID (integer)
`ODOO_PASSWORD`	User password

Creating an Odoo Integration — 4 Steps

1 Define interface in `Application.UseCases/Repositories/Integration/`:

```
public interface ITimecardOdooIntegration
{
    Task<IEnumerable<TimecardDTO>> GetTimecardsAsync(string from, string to);
    Task<int> PostTimecardAsync(TimecardDTO payload);
}
```

2 Implement in `Integration.Odoo/Implementations/` — always `internal`:

```
internal class TimecardIntegration(IOdooActions odoo) : ITimecardOdooIntegration
{
    public Task<IEnumerable<TimecardDTO>> GetTimecardsAsync(string from, string to)
        => odoo.SearchReadAsync<TimecardDTO>("time.card.line",
               [["time", "≥", from], ["time", "≤", to]],
               ["time", "biometric_name"]);

    public Task<int> PostTimecardAsync(TimecardDTO dto)
        => odoo.CreateAsync("time.card.line", dto);
}
```

3 Register → `services.TryAddTransient<ITimecardOdooIntegration, TimecardIntegration>()`

4 Wire up MediatR query/command in `Application.UseCases` — same as any feature.

📏 Code Conventions

Part 11

Naming · patterns · do's and don'ts

PART 11 OF 12

📏 Naming Conventions

Type	Pattern	Example
Service interface	`IXxxService`	`IOrderService`
Service implementation	`XxxService`	`OrderService`
Repository interface	`IXxxRepository`	`IOrderRepository`
Repository implementation	`XxxRepository`	`OrderRepository`
MediatR command	`XxxCommand`	`CreateOrderCommand`
MediatR query	`XxxQuery`	`GetOrderByIdQuery`
MediatR handler	`XxxHandler`	`CreateOrderHandler`
DTO	`XxxDto`	`OrderDto`
Domain entity	`XxxDEM`	`OrderDEM`
Value object	`XxxVO`	`MoneyVO`
Blazor page	`XxxPage`	`OrderListPage`
ViewModel	`XxxVM`	`OrderVM`

📏 Error Handling Strategy

Error Type	Strategy	Example
Expected / business failure	Return `null`, `bool`, or result wrapper	"Order not found" → `null`
Unexpected / system error	Throw an exception	DB connection failure → throw

```
// ✅ Expected — return null (caller handles "not found")
public async Task<OrderDto?> GetOrderAsync(Guid id)
    => (await _repo.GetByIdAsync(id))?.Adapt<OrderDto>();

// ✅ Unexpected — throw
public async Task<OrderDto> GetRequiredOrderAsync(Guid id)
{
    var order = await _repo.GetByIdAsync(id)
        ?? throw new InvalidOperationException($"Order {id} not found.");
    return order.Adapt<OrderDto>();
}

// ❌ Don't throw for normal business cases
if (order == null) throw new Exception("Order not found"); // ❌
```

🚫 Things You Must Never Do

❌ Never	Why
`.Result` · `.Wait()` · `.GetAwaiter().GetResult()`	Deadlocks in Blazor Server
Raw SQL with string interpolation	SQL injection
Return Domain entities across layer boundaries	Breaks encapsulation
Hardcode secrets, connection strings, keys	Security — will be committed
`dynamic` or anonymous types across layers	Breaks type safety
Inject `IMediator` or `AppDbContext` into components	Skips the pipeline
Re-inject services already in `BaseComponent`	Redundant
EF attributes on Domain entities	Couples Domain to Infrastructure

📏 Layer Boundary Rules

✅ Presentation → can use → Application
✅ Application → can use → Domain
✅ Application → can use → Infrastructure (via interfaces)
✅ Infrastructure → implements → Application interfaces

❌ Domain → must NOT use → Application
❌ Domain → must NOT use → Infrastructure
❌ Application → must NOT use → Presentation
❌ Presentation → must NOT use → Infrastructure directly

Quick Reference — "Where Does This Go?"

I need to…	It belongs in…
Define what an Order looks like	`Domain.Entities`
Write a business rule	`Domain.Entities`
Create a "Place Order" feature	`Application.UseCases/Commands/Orders/`
Write the EF/SQL query	`Database.MsSql`
Create a page for placing orders	`Web.BlazorServer/Components/Pages/`
Call SAP for order data	`Integration.Sap`
Call NetSuite for order data	`Integration.NetSuite`
Call Odoo for order data	`Integration.Odoo`

🏁 Debts & Building Locally

Part 12

Known issues + day-one setup

PART 12 OF 12

⚠ Known Architectural Debts

> Do not fix any of these without explicit instruction from a senior engineer or tech lead.

#	Issue	Affects You?	Action
1	Domain references MediatR	Rarely	Use domain events normally
2	Inconsistent aggregate root enforcement	⚠ Sometimes	Verify per-aggregate before writing
3	Generic + specific repositories coexist	Minor	Follow current rules
4	Mixed error handling in old handlers	Minor	Apply new rules to new code only
5	Web layer dispatch coordination (Pipeline A)	None	Follow Pipeline A as-is

These are documented so you're **aware** — not so you can fix them.

Write new code following the documented rules. Leave old code alone.

🔨 Building & Running Locally

Prerequisites

.NET 8 SDK · SQL Server (local or Docker) · VS / VS Code / Rider

```
# Set env var BEFORE running (never put in appsettings.json)
# PowerShell
$env:ConnectionStrings__DefaultConnection = "Server=localhost;Database=YourDB;Trusted_Connection=True;"
# macOS / Linux
export ConnectionStrings__DefaultConnection="Server=localhost;Database=YourDB;Trusted_Connection=True;"
```

```
dotnet build
dotnet run --project Web.BlazorServer
```

```
# Apply migrations (run from solution root)
dotnet ef database update \
  --project Database.MsSql --startup-project Web.BlazorServer
```

🔧 Troubleshooting

Problem	Check
App won't start	Is `ConnectionStrings__DefaultConnection` set?
Database connection error	SQL Server running? Connection string correct?
Migration errors	Running from solution root? Database exists?
Login fails silently	DB seeded? Check `AppDbSeeding.cs`
Cookie auth not working	`CookieAuthenticationDefaults.AuthenticationScheme` registered in `Program.cs`?
New permission not working	Did user **log out and back in**? Claims refresh on login only

🏆 Day Summary

Layer	Responsibility	Key Rule
Domain	Business rules & data shapes	Private setters · factory methods · guards
Application	Use case orchestration	Commands/Queries · Handlers · DTOs
Infrastructure	Data access & external calls	Repositories · EF Core · SAP · NetSuite · Odoo
Presentation	UI & user interaction	Web Repos → Handlers → MediatR

```
Component → WebRepository → WebHandler → IMediator.Send()
  → AppHandler → IRepository → DbContext → SQL Server
```

> Every layer has a job. Don't skip layers.

VCCA Architecture Training — Documentation Summary

Maintained by Ian Nicolas Antonio · Engineered with Charles Maverick Herrera & Arnel De Asas · 2026